

B3: Access Control

Reading a space takes a **space credential**, and the space authority is the only party that issues one. Getting there involves three tokens: the delegation token, the client attestation, and the space credential itself. All three are JWTs, but they're signed by different parties, addressed to different recipients, and prove different things. Once you can keep the three straight, the rest of this section is bookkeeping.

The authority weighs two questions when deciding whether to issue, and each has its own token:

Reading a space is gated by a **space credential** issued by the space authority. The authority issues one based on two axes:

- **which user** is being acted for: established by a **delegation token** minted by the user's PDS
- **which application** is acting: established by a **client attestation** signed by the application itself

Permissioned Data proposal, "Access control"

The two axes don't apply equally:

The delegation token is always required. The client attestation is required only when a space gates on client app identity.

Permissioned Data proposal, "Access control"

The exchange itself is simple: the application gets a delegation token from a user's PDS, then presents it (with its client attestation, if the space wants one) to the space authority and gets a credential back. Whether the authority actually says yes is a policy question the protocol deliberately leaves open:

The authority decides whether to issue the credential. The protocol does not define the decision procedure (the policies of the PDS's space-management implementation are described under [simplespace](#)).

Permissioned Data proposal, "Access control"

?

The protocol carries no member list either. It specifies the tokens and the exchange; the policy behind them (member lists, public access, dynamic checks by a managing app) belongs to space-management implementations such as `com.atproto.simplespace`, which sit above the protocol.

An application can find out whether a given space demands an attestation. Per the proposal: "Some spaces do not require a client attestation. This can be detected by querying the space configuration, or by simply making a request for a credential without an attestation and seeing whether an error is returned."

The delegation token

A **delegation token** proves that an application is acting on a user's behalf when it asks an authority for a space credential. The user's PDS mints it, requested through `com.atproto.space.getDelegationToken`. It asserts only the delegation from user to application. Whether the user is a member of the space is the authority's determination, and the delegation token says nothing about it. The token is single-use, short-lived (default 60 seconds), and addressed (`aud`) to the space authority.

Permissioned Data proposal, "Delegation token"

The token asserts exactly one thing: this user delegated to some application. It does not assert space membership, which is the authority's call. And it does not identify the application either, which surprises people the first time. More on that below.

Prerequisite: a covering OAuth scope

An application can't request a delegation token for arbitrary spaces. Per the proposal, "A client app session can only request a delegation token for a space it has been authorized to access. The application must hold a covering `space:` OAuth scope (specifically a `read` grant, which confers access to `getDelegationToken`)." This is why OAuth consent is step one of the credential flow: the user's grant is what makes the PDS willing to mint delegation tokens for the space at all. The narrower `read_self` grant does *not* confer `getDelegationToken`; the B6 lesson on OAuth scopes covers the difference.

Structure: a distinct credential class

A delegation token is structurally similar to an atproto service auth token, but the proposal lists three deliberate differences that make it its own credential class rather than an interchangeable one:

- The `typ` field in the header is set to `atproto-space-delegation+jwt`.
- It does not include an `lxm` claim.
- It is bound to a target space through the `sub` claim.

Permissioned Data proposal, "Delegation token"

So where a service auth token names a specific method via `lxm`, the delegation token has no `lxm` at all; its `sub` claim carries the URI of the space being requested instead. The distinct `typ` value prevents one token class from being replayed as the other.

The proposal's example JWT header and payload (before base64url encoding and signing):

```
{
  "typ": "atproto-space-delegation+jwt",
  "alg": "ES256K", // or ES256
  "kid": "#atproto" // Key Identifier – MUST be "#atproto"
}
{
  "iss": "did:example:user_did", // User DID
  "sub": "at://did:example:space_did/space/com.example.space_type/space_key", // Space
  "aud": "did:example:space_did#atproto_space_host", // Space host (service fragment o
  "iat": 1738368000, // Issued-at (unix seconds)
  "exp": 1738368060, // iat + 60 (60 seconds)
  "jti": "f47ac10b58cc4372a5670e02b2c3d479" // random nonce
}
```

Reading the claims:

- `iss`: the **user's** DID. The user is the party doing the delegating.
- `sub`: the space URI (`at://{spaceDid}/space/{spaceType}/{skey}`) of the space being requested. This is the space binding that replaces `lxm`.
- `aud`: the space host, expressed as the service fragment of the authority DID. The token is addressed to one specific recipient and is useless anywhere else.
- `exp`: `iat + 60`, the 60-second default lifetime. Between that, single-use semantics, and the `jti` nonce, the token lives just long enough to be exchanged once.
- `kid`: MUST be `#atproto`, the account's signing key.

The signature for the delegation token is computed using the regular JWT process, using the account's signing key.

Permissioned Data proposal, "Delegation token"

What the delegation token does not say

The delegation token asserts only the user-to-app delegation; it says nothing about which application is acting. App identity is established independently by the client attestation, which the authority verifies itself. The two are presented together but signed by different parties and evaluated independently.

Permissioned Data proposal, "Delegation token"

So you can't tell which application is asking by inspecting a delegation token. There is no client identifier anywhere in it. When the authority needs to know which app is asking, that answer has to come from the second token.

Client attestation

A **client attestation** is a short-lived, single-use JWT that the application presents to the space authority alongside the delegation token to identify itself, when the space requires it. It is structurally a **private_key_jwt** client assertion, the same shape an atproto confidential client already presents to its authorization server, but addressed to the space authority rather than to the PDS.

Permissioned Data proposal, "Client attestation"

Where the delegation token is signed by the user's account key via the PDS, the attestation is signed by the **application's own client authentication key**. Its one job is proving app identity to the authority. It's required only when the space gates on client app identity (in **simplespace** terms, an **appAccess** of **#allowList** rather than the default **#open**). Whether it's needed is a property of the space, not of the client.

The proposal's example:

```
{
  "typ": "atproto-client-attestation+jwt",
  "alg": "ES256",
  "kid": "key-1" // Key id from the client's published JWKS
}
{
  "iss": "https://app.example.com/client-metadata.json", // The client_id
  "sub": "https://app.example.com/client-metadata.json", // The client_id (== iss)
  "aud": "did:example:space_did#atproto_space_host", // Space host being asked for a c
  "iat": 1738368000, // Issued-at (unix seconds)
  "exp": 1738368060, // short-lived
  "jti": "b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8" // random nonce, replay protection
}
```

Both **iss** and **sub** are the **client_id**: a URL pointing at the client's metadata document, not a DID. The **kid** names a key from the client's published JWKS, and the **jti** nonce provides replay protection.

How the authority verifies it

The authority verifies the attestation by resolving **iss** (the **client_id**) to the client's **client-metadata.json**, fetching its published JWKS (**jwks** or **jwks_uri**), and verifying the signature

against the key in that JWKS identified by the attestation's `kid`.

Permissioned Data proposal, "Client attestation"

The authority does all of this on its own. No PDS is consulted, no DID document is resolved, and there's no shared secret or prior registration with the authority. Everything it needs is public key material the client already publishes at its `client_id`. That's also what makes app gating enforceable: when a `simplespace` space uses an app allow-list, the list is checked against the *attested* `client_id` (the `iss` of the verified attestation), not a self-reported one.

The space credential

A **space credential** is the token an application presents to a repo host to read a permissioned repo within a space. The authority mints it in exchange for a delegation token, requested through `com.atproto.space.getSpaceCredential`. It is short-lived (default 2 hours) and signed by the space authority's signing key, so any repo host can verify it against the authority's key without contacting the authority.

Permissioned Data proposal, "Space credential"

This is the token that actually opens doors. An application holding a space credential can call the read and sync methods (`getRecord`, `listRecords`, `getBlob`, `getLatestCommit`, `getRepo`, `listRepoOps`) against every repo host serving a repo in the space, including hosts of members the application has no OAuth relationship with. Verification is offline: the credential is signed by the authority's signing key, so a repo host checks it against the authority's published key material and never has to phone the authority.

A space credential is multi-use. A single credential is intended to be reused across every repo host serving a repo in the space as well as against a given host for repeated requests, until it expires.

Permissioned Data proposal, "Space credential"

The proposal defines it by contrast with the delegation token:

A space credential resembles a space delegation token, differing in these ways:

- The `typ` field in the header is set to `atproto-space-credential+jwt`.
- It is signed by the space authority rather than the user.
- It has no `aud`: it is presented to any repo host serving a repo in the space, not to a single recipient.

Permissioned Data proposal, "Space credential"

The missing **aud** is a deliberate choice, not an oversight. A token meant for reuse across an arbitrary set of repo hosts can't be addressed to any single one of them. The proposal's example:

```
{
  "typ": "atproto-space-credential+jwt",
  "alg": "ES256K", // or ES256
  "kid": "#atproto_space" // Key Identifier - "#atproto_space" or "#atproto"
}
{
  "iss": "did:example:space_did", // Space authority DID
  "sub": "at://did:example:space_did/space/com.example.space_type/space_key", // Space
  "iat": 1738368000, // Issued-at (unix seconds)
  "exp": 1738375200, // iat + 7200 (2 hours)
  "jti": "9f8e7d6c5b4a3210fedcba9876543210" // random nonce
}
```

Here **iss** is the **space authority's** DID (in the delegation token it was the user's), **sub** is again the space URI, there is no **aud**, and **exp** is **iat + 7200**, the 2-hour default. The **kid** is **#atproto_space** or **#atproto**: the authority's dedicated space signing key from its DID document, falling back to its regular **#atproto** key when no dedicated one is published. Per the proposal, "The signature for the space credential is computed using the regular JWT process, using the space authority's signing key."

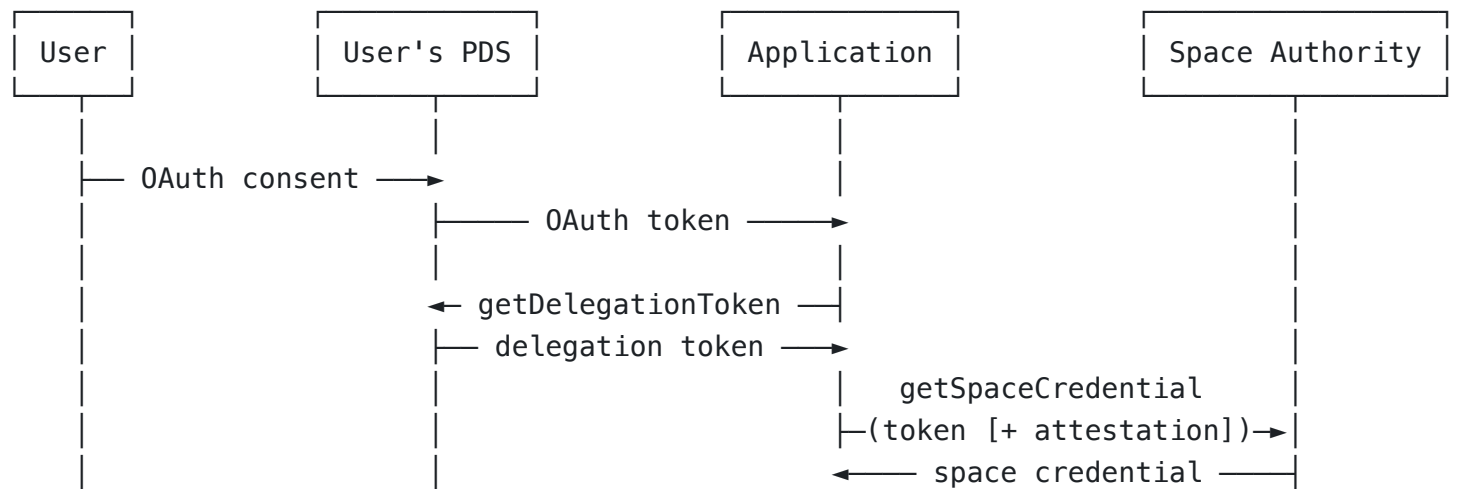
The three tokens side by side

	Delegation token	Client attestation	Space credential
typ	atproto-space-delegation+jwt	atproto-client-attestation+jwt	atproto-space-credential+jwt
Signed by	User's PDS, with the account's signing key (kid MUST be #atproto)	The application, with its client authentication key (kid from its published JWKS)	Space authority's signing key (kid is #atproto_space or #atproto)
iss	User DID	client_id (URL of client-metadata.json)	Space authority DID
sub	Space URI being requested	client_id (== iss)	Space URI the credential reads
aud	Space host (service fragment of the authority DID)	Space host being asked for a credential	<i>None</i> ; presented to any repo host in the space
Use	Single-use	Single-use	Multi-use, across hosts and requests

Default lifetime	60 seconds	Short-lived	2 hours
Presented to	Space authority	Space authority (alongside the delegation token)	Repo hosts
Proves	User-to-app delegation	App identity	Read access to the space

The credential flow

The proposal's flow diagram, reproduced verbatim:



And the four steps, exactly as the proposal enumerates them:

1. The user authorizes the application via OAuth.
2. The application calls `com.atproto.space.getDelegationToken` on the user's PDS, receiving a delegation token.
3. The application presents the delegation token to the space authority via `com.atproto.space.getSpaceCredential`, adding its own client attestation if the space gates on client app identity. The authority verifies what it received and, on authorization, returns a space credential.
4. The application reads the repo from each member's repo host with the credential.

Permissioned Data proposal, "Credential flow"

The ordering is forced by the dependencies between steps. The application can't call `getDelegationToken` until the user has granted it a covering `space:` scope via OAuth, so step 1 comes before step 2. The authority has nothing to evaluate until it receives a delegation token, so step 2 comes before step 3. And repo hosts accept the space credential, not the delegation token or the attestation, so reading comes last. Keep the services straight as well: `getDelegationToken` is served by the **user's PDS**; `getSpaceCredential` is served by the **space authority** (its space host).

One credential serves many users

The space credential grants whole-space read access and carries no per-user binding for its use at repo hosts, so an application's credential bookkeeping stays light:

An application serving several users of a space does not necessarily need to maintain a space credential for each user. It may obtain its credential using any one user's session. When it loses all OAuth sessions for a space, it can no longer renew the credential and loses access.

Permissioned Data proposal, "Credential flow"

A forum client serving hundreds of members of the same space can hold a single credential, minted through whichever user's session is convenient, and reuse it against every member's repo host until it expires. The flip side is the revocation story built into that sentence: an app's continued access depends on holding at least one live OAuth session for the space. Lose them all and there's no way to renew, so access lapses when the current credential expires. And since credentials are short-lived by design, the space authority can cut off any reader simply by declining to issue the next one.

You should now be able to explain

- The two axes a space authority weighs when issuing a space credential, and which token establishes each
- Which of the two tokens is always required, and when a client attestation is needed
- How an application can find out whether a space requires an attestation
- Who mints and signs the delegation token, through which XRPC method, and with which key (`kid`)
- The delegation token's usage properties: single-use, 60-second default lifetime, `aud` addressed to the space authority
- What the delegation token asserts, and what it deliberately does not (space membership, app identity)
- The three ways a delegation token differs from a service auth token (`typ`, no `lxm`, space-bound `sub`)
- Why the OAuth `space:` scope (a `read` grant) is a prerequisite for requesting a delegation token
- What a client attestation is structurally, who signs it, and what its `iss/sub` contain
- How the authority verifies an attestation: `iss` to `client-metadata.json` to JWKS to the key named by `kid`
- What a space credential is for, who signs it, its 2-hour default lifetime, and why it is multi-use
- Why the space credential has no `aud`, and how repo hosts verify it without contacting the authority
- Which token an application presents to a repo host to read another member's repo
- The four steps of the credential flow, in order, and which service serves each call
- Why an app serving many users of a space can hold a single credential, and what happens when it loses all OAuth sessions for the space

- That the protocol specifies the tokens and exchange but not the authorization decision procedure, which belongs to space-management implementations like **simplespace**